PRESENTATION STUDY GUIDE / 01

Understand the project

A beginner-friendly guide to presenting the CartPole reinforcement learning comparison.

The central question. If different methods can learn to balance a pole, how much practice, time and complexity does each method need?

The balancing task

Imagine balancing a broom upright on your palm. You move your hand to keep it underneath the broom. CartPole simulates a similar task. A pole sits on a cart, and the computer pushes the cart left or right to keep it upright.

The agent observes four numbers: **cart position, cart velocity, pole angle and pole angular velocity**. Velocity describes speed and direction. Angular velocity describes how fast the pole rotates.

Each step gives one reward point. An episode ends when the pole falls beyond about 12 degrees, the cart moves outside its permitted region, or the 500-step limit is reached. **The maximum episode score is 500.**

What reinforcement learning means

The program learns through actions and rewards. It is not given a list of correct actions. Its loop is: observe the situation, choose an action, receive a reward and the next observation, then learn from the experience.

Term	Meaning in this project
Agent / environment	The learning program / the CartPole simulator.
State / action	The current four-number observation / push left or right.
Reward / return	Immediate feedback / rewards accumulated over time.
Episode / policy	One complete attempt / the rule for choosing actions.
Training / evaluation	Practice that changes the agent / testing without updates.
Hyperparameter / seed	A setting chosen before training / a number controlling randomness.

A fixed rule based only on pole angle misses information. The pole might already be moving back toward the centre. The agent needs to consider the full observation.

PRESENTATION STUDY GUIDE / 02

Tabular Q-learning

Tabular Q-learning learns a lookup table. For each discretised state, it stores one estimated future-reward score for each action. The letter Q refers to that action value.

Example situation	Value of left	Value of right
A particular state	70	90

The greedy decision is right because 90 is larger than 70. These are illustrative values, not measurements from a run.

Why the table needs bins

CartPole observations are continuous. An angle could be 0.01, 0.011 or countless other values. A finite table cannot have a separate row for every possible observation. Your code divides each observation dimension into bins. Nearby observations share a cell.

Selected bins: **4 x 4 x 8 x 8 = 1,024 state cells.**
Two action values per cell give **2,048 table entries.**

The two velocity dimensions are clipped to finite limits before binning. Coarse bins merge different physical situations. Fine bins preserve more detail but create many more entries to learn.

How the table learns

$$Q(s,a) \leftarrow Q(s,a) + \alpha \times [r + \gamma \times \max Q(s',a') - Q(s,a)]$$

Say the equation in words: **adjust the old action score toward the reward just received plus the estimated value of the next state.** On a genuine terminal transition, the future-value term is zero.

Symbol	Meaning
s, a, r, s'	Current state, chosen action, received reward, next state.
alpha	Learning rate. It controls the size of the score change.
gamma	Discount factor. It controls the weight of future rewards.
max Q(s',a')	The highest estimated action value in the next state.

Exploration versus exploitation

The epsilon-greedy rule sometimes chooses a random action to explore. Otherwise, it chooses a highest-scoring action. Your selected setup decreases epsilon from 1.0 to 0.05 over 10,000 steps.

Presentation sentence. "Tabular Q-learning is a fast lookup-table method, but binning limits how precisely it represents the continuous task."

Source: rlcompare/agents/tabular.py; configs/tuned.yaml; rlcompare/config.py.

PRESENTATION STUDY GUIDE / 03

DQN learns action values with a network

DQN means Deep Q-Network. It has the same basic goal as tabular Q-learning: estimate the future reward associated with each action. It replaces the table with a neural network.

> **4 observation numbers**
> then 128 hidden neurons
> then 128 hidden neurons
> then **2 action values**, one for left and one for right.

The network can learn patterns across observations without explicitly assigning every observation to a table bin. Its hidden layers use ReLU activations.

The replay buffer stores experience

Each stored transition contains the current observation, action, reward, next observation and termination flag. DQN samples random batches from this memory.

Replaying experience lets the agent reuse data. Random batches also reduce the temporal correlation between neighbouring examples. The selected buffer holds up to 50,000 transitions and the batch size is 32.

The target network stabilises the target

The online network is the network being trained. A separate target network is a copy used to calculate the next-state value in the learning target. It remains fixed between refreshes.

> "The network learns against a target that stays fixed for a while. We periodically refresh that target as learning improves."

Your selected target network refreshes after every 500 learning updates. Without this separation, changing the prediction also changes the target used to train it.

What one DQN update does

DQN samples a replay batch, predicts the values of the actions that were taken, and builds targets from the observed rewards plus the target network's next-state estimates. It then adjusts the online network to reduce prediction error.

The code uses Huber loss, which is less sensitive to large errors than squared error. Adam adjusts the weights. Gradient clipping limits the size of a gradient. Training starts once the buffer contains 1,000 experiences.

Selected setting	Value
Learning rate	0.0003
Discount factor	0.99
Exploration	Epsilon-greedy, declining to 0.05.
Training frequency	One replay-batch update per step after warm-up.

Presentation sentence. "DQN learns action values with a neural network, reuses experience through replay, and uses a target network to stabilise learning."

Source: rlcompare/agents/dqn.py; configs/tuned.yaml; rlcompare/config.py.

PRESENTATION STUDY GUIDE / 04

REINFORCE learns the policy directly

REINFORCE directly learns action probabilities. For example, its policy might give left a probability of 0.3 and right a probability of 0.7. These numbers are an illustration.

During training, it samples actions using these probabilities. During evaluation, your implementation chooses the highest-probability action. It does not need an epsilon-greedy rule.

Learning from a completed episode

After an episode, the method calculates the discounted future reward following each action. It adjusts the policy so that actions associated with better-than-expected outcomes become more likely.

Advantage = observed discounted future return - baseline prediction.

Expected return	Observed return	Advantage	Signal
80	100	+20	Increase the action's probability.
80	60	-20	Decrease the action's probability.

What the baseline contributes

A separate value network predicts the expected discounted return from the current state. Subtracting this baseline gives the policy a comparison against expectations and can reduce variability in the learning signal.

The baseline learns by predicting the discounted returns observed in episodes. Your code trains the policy and baseline separately. Returns are not standardised by their standard deviation, so the baseline ablation does not also change that scaling.

Why this remains policy-based

"Its main objective directly improves action probabilities. The value network supports that learning as a baseline."

Selected setting	Value
Policy and baseline architecture	Two hidden layers of 128 neurons in each network.
Policy learning rate	0.0003
Baseline learning rate	0.003
Discount factor	0.999
Episodes per update	1

Gamma gives future rewards less weight than immediate rewards. A value near one still places substantial weight on the future. Training uses discounted returns, while reported evaluation scores are ordinary sums of reward points.

Remember the difference. Q-learning and DQN estimate action values. REINFORCE directly adjusts the action-selection policy.

Source: rlcompare/agents/reinforce.py; rlcompare/utils.py; configs/tuned.yaml.

PRESENTATION STUDY GUIDE / 05

A controlled comparison

The contribution is a controlled comparison of existing algorithms. All methods use one shared training loop and the same evaluation procedure.

Setting	Value
Training budget	100,000 environment steps per run.
Main comparison	10 seeds per method.
Evaluation	Every 2,000 training steps, plus a final checkpoint.
Episodes per checkpoint	10, with exploration disabled.
Tuning budget	12 trials per method, using 3 tuning seeds per trial.

A weak agent may fail after 20 steps, while a strong one lasts 500. Equal episode counts would give them different amounts of experience. Equal training-step budgets avoid that problem.

Different seeds vary random initialisation and experience. Tuning seeds differ from the main comparison seeds. Evaluation uses a fixed common set of episode seeds. Tuning chooses settings by mean area under the evaluation curve, rewarding both high scores and early improvement.

Stage	Runs
Tuning: 3 methods x 12 trials x 3 seeds	108
Main comparison: 3 methods x 10 seeds	30
Sweeps: 3 methods x 5 settings x 5 seeds	75
Ablations: 2 removals x 10 seeds	20
Total	233

Reported headline results

Method	Steps to first solve	Final IQM	Time	Never solved
Tabular Q-learning	36,000	226	16 s	10%
DQN	55,000	486	214 s	0%
REINFORCE + baseline	18,000	500	46 s	0%

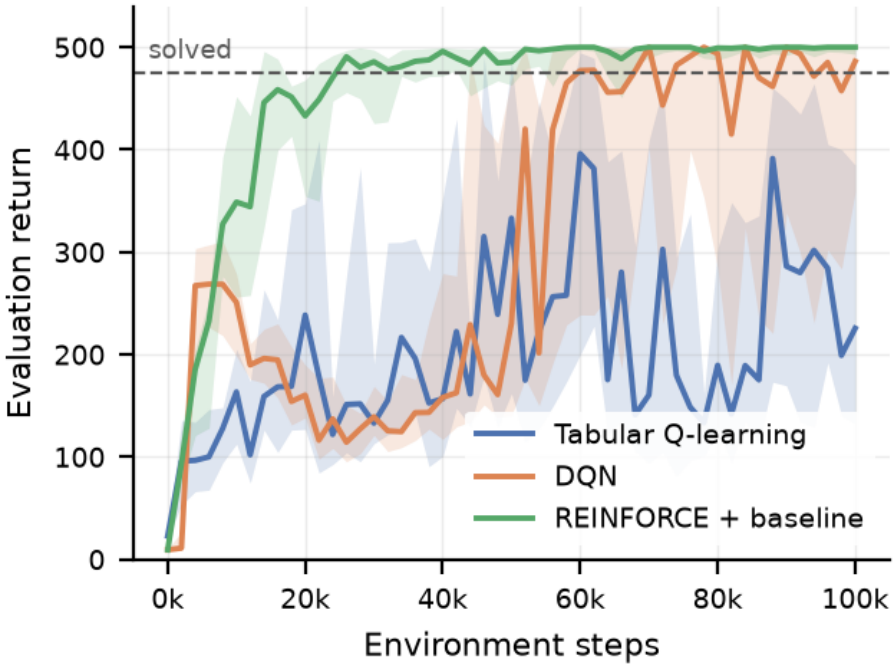
Steps to solve is the median among runs that crossed the threshold. Here, "solved" means an average of at least 475 across ten evaluation episodes at one checkpoint. It does not require maintaining that score.

Practice efficiency and running time differ. REINFORCE reaches the threshold with the fewest interactions. Tabular Q-learning runs fastest, but has much poorer final performance. DQN reaches a strong final score with the longest reported running time.

Source: configs/study.yaml; paper/tables/tradeoff.tex; rlcompare/experiments/runner.py.

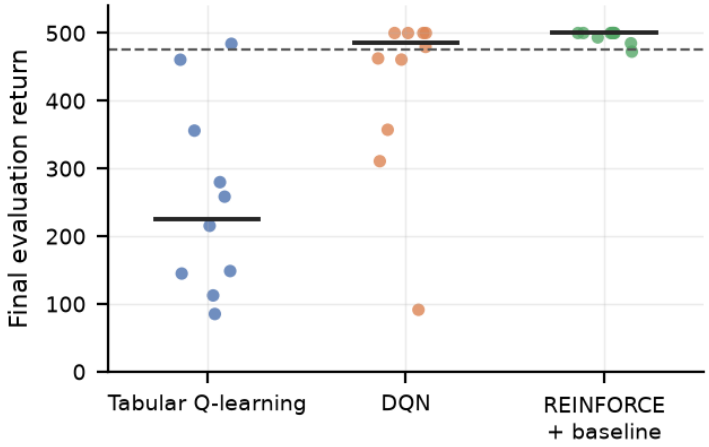
PRESENTATION STUDY GUIDE / 06

Read the results visually



The horizontal axis shows training interactions. The vertical axis shows evaluation score. An earlier rise means fewer interactions are needed. A curve near 500 performs strongly. A later drop shows that further learning can reduce performance.

The green REINFORCE curve improves early and stays near the maximum. DQN improves later. Tabular Q-learning fluctuates and finishes much lower.



Each dot shows one run's final result. This exposes individual failures that an aggregate can hide. A 0% never-solved rate means all seeds reached the threshold at least once, even if some finished below it.

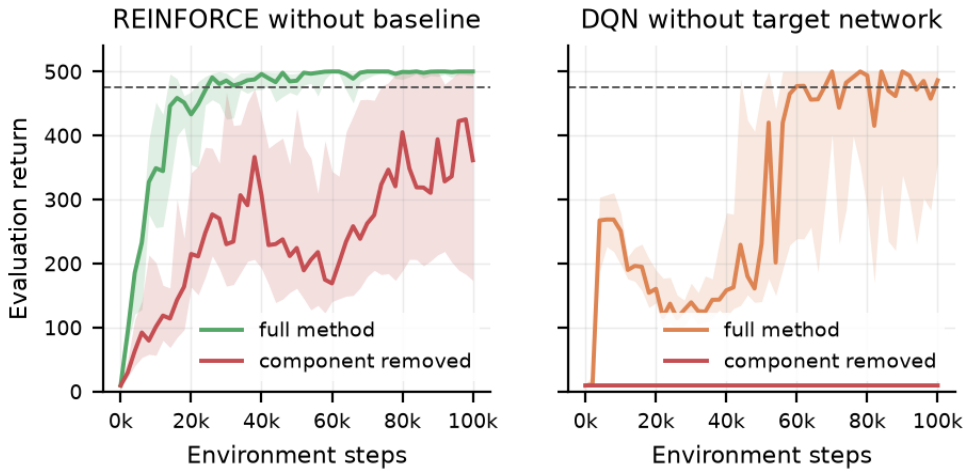
IQM summarises the middle portion of the seed results to reduce the influence of extreme outcomes. The shaded bands are **95% bootstrap confidence intervals**, estimated by repeatedly resampling seed results. Wide bands mean more uncertainty in the aggregate. They do not describe where 95% of future runs will fall.

Source: paper/figures/learning_curves.png; paper/figures/seed_scatter.png; rlcompare/analysis/stats.py.

PRESENTATION STUDY GUIDE / 07

Test components and settings

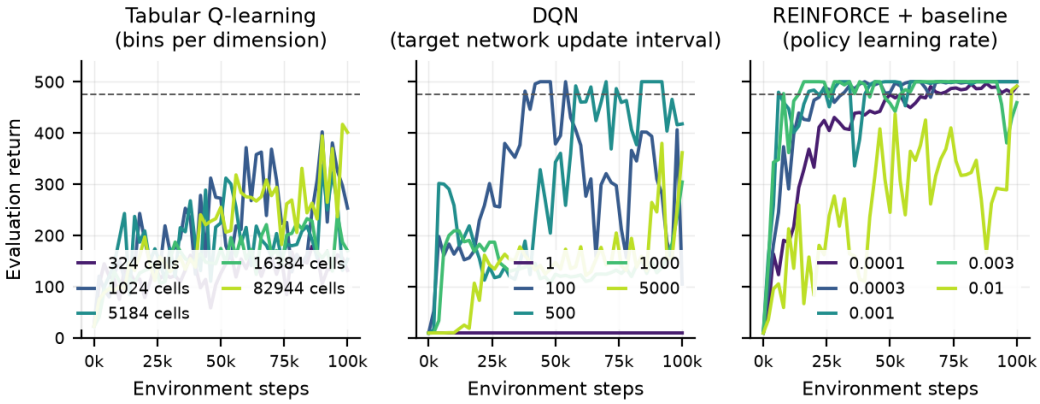
An **ablation** removes one component while keeping the other selected settings. It asks what that component contributes under this configuration.



Version	Final aggregate return
Full DQN	486
DQN without target network	9
Full REINFORCE	500
REINFORCE without baseline	361

Under the selected settings, removing the target network causes DQN to fail. Removing the baseline reduces REINFORCE's performance, although it still learns. This does not prove that every possible DQN without a target network must fail.

Sensitivity sweeps ask a different question



A sweep changes one setting through several values. Your study varies tabular bin counts, the DQN target-refresh interval and the REINFORCE learning rate. It asks how strongly the result depends on each choice.

The tabular results vary with the grid. DQN is sensitive to target refresh timing. REINFORCE reaches high final aggregate returns across several tested learning rates. A sweep uses five seeds per setting, so its aggregate need not equal the ten-seed main result.

Source: paper/tables/ablations.tex; paper/figures/ablations.png; paper/figures/sweeps.png; configs/study.yaml.

PRESENTATION STUDY GUIDE / 08

Know the implementation and its limits

The experiment follows this sequence: choose the study settings, tune hyperparameters, run training and evaluation, save measurements, calculate statistics, and generate tables and figures.

File	Responsibility
run.py	Starts tuning, the experiment matrix and analysis.
configs/study.yaml	Defines budgets, seeds, search spaces and experiments.
configs/tuned.yaml	Stores the selected hyperparameters.
rlcompare/agents/tabular.py	Implements binning and the Q-table.
rlcompare/agents/dqn.py	Implements DQN and replay memory.
rlcompare/agents/reinforce.py	Implements the policy and baseline.
rlcompare/experiments/runner.py	Runs the shared training and evaluation loop.
rlcompare/analysis/stats.py	Calculates aggregates and uncertainty.

What the shared loop does

The loop observes the state, asks the agent for an action, steps the simulator and passes the experience back to the agent. Each algorithm learns differently from that experience. At evaluation checkpoints, learning pauses and the current policy is tested.

The loop distinguishes genuine termination from the 500-step time limit. The value-based methods bootstrap through a time-limit truncation, but not through genuine failure.

Accuracy notes for your presentation

Keep the conclusion local. These results apply to the tested CartPole setup. They do not establish an overall ranking for reinforcement learning. Dense rewards and a small observation space plausibly help REINFORCE here.

Use the configuration's grid. The selected bins are 4, 4, 8, 8. The paper calls this the coarsest candidate, but the search also included the coarser 3, 3, 6, 6 grid.

Do not equate update counts with computation. A table update, a replay-batch gradient update and an episode-level REINFORCE update are different operations. One REINFORCE counter increment includes separate policy and baseline learning.

Interpret the timing correctly. It includes evaluation and depends on the machine. The training budget counts training interactions, not evaluation interactions.

State the evidence limits. The repository contains the paper, figures and generated tables, but no raw results folder in this checkout. This guide reports those figures without claiming a fresh reproduction.

The study's main limitations are one environment, ten main seeds, a small tuning budget and implementation-dependent complexity counts. Binning is a plausible contributor to tabular instability, not a demonstrated sole cause.

PRESENTATION STUDY GUIDE / 09

Practise your presentation

A short script to say aloud

"Our project compares tabular Q-learning, DQN and REINFORCE with a learned baseline on CartPole.

The task is to push a cart left or right to keep a pole upright. The agent receives one reward per step, with a maximum episode score of 500.

Tabular Q-learning stores action values in a discretised table. DQN predicts action values using a neural network. REINFORCE directly learns action probabilities.

We use the same training loop and a budget of 100,000 training steps per run. We compare ten seeds per method and give each method the same number of tuning trials.

In our reported results, tabular Q-learning runs fastest but has weaker final performance. DQN achieves a strong final score with greater computational cost. REINFORCE reaches the solved threshold earliest and achieves the highest aggregate final score.

Our ablations show that the target network is important for DQN under these settings, while the baseline improves REINFORCE. Our conclusion is that the algorithm should match the task and available resources. These findings need testing on other environments."

Likely questions and short answers

Question	Answer
Why these three methods?	They compare a value table, a neural value function and direct policy learning.
Why does the table need bins?	A finite table cannot directly represent every continuous observation.
Why use replay memory?	To reuse experience and train on less temporally correlated examples.
Why use a target network?	To keep the learning target more stable between refreshes.
Why use a baseline?	To compare outcomes against expectations and reduce learning-signal variability.
Why repeat with different seeds?	To measure variation caused by randomness.
Why solve and later get worse?	Further training changes the policy. First success does not guarantee retained performance.
Is REINFORCE always best?	This study supports results for these settings on CartPole.
What is the contribution?	A controlled comparison of performance, practice cost, computation and implementation cost.
What would you do next?	Test harder environments, including sparse rewards or higher-dimensional observations.